Audit: fheCRDT Soundness and Empirical Performance

Automated Research Agent
Lux Industries
research@lux.network

2026-04-12

Abstract

We audit the fheCRDT paper [?] and its reference implementation in `github.com/luxfi/fhe`. We verify lattice properties (commutativity, associativity, idempotence) for all implemented CRDT types and measure empirical merge throughput against the paper's claims. Key findings: (1) the LWW-Register implementation is correct and preserves all three lattice properties under encryption; (2) G-Counter, PN-Counter, G-Set, OR-Set, and RGA have *no Go implementation*—only the LWW-Register exists as executable code; (3) measured throughput is **3–4 orders of magnitude below** the paper's claims (0.24 LWW merges/sec vs. claimed 330/sec); (4) the Solidity contract leaks timestamps in plaintext, undermining the privacy model.

1 Scope and Method

1.1 Materials Examined

- `cmd/crdt/main.go` — LWW-Register demo (184 LOC)
- `fhe.go`, `evaluator.go`, `encryptor.go`, `decryptor.go` — core FHE boolean gate engine
- `examples/encrypted-crdt/contracts/EncryptedCRDT.sol` — Solidity LWW-Register + OR-Set
- `main.tex` — the fheCRDT paper
- `hanzo/base/crdt/` — plaintext CRDT baseline (GCounter, PNCounter, LWWRegister, ORSet, MVRegister, RGA)

1.2 Test Environment

CPU	Apple M1 Max (10-core, arm64)
RAM	64 GB
OS	Darwin 25.4.0
Go	1.26.0
FHE params	PN10QP27 (N=1024, Q=134215681, 128-bit security)

1.3 Method

For each CRDT type claimed in the paper, we:

1. Searched the codebase for a Go implementation.
2. Where code exists, wrote property-based tests for the three lattice laws plus observational equivalence to plaintext.
3. Ran Go benchmarks (`-benchtime=3s`) measuring wall-clock time per merge.
4. Compared against plaintext CRDT merge cost from `hanzo/base/crdt/`.

2 Soundness Findings

2.1 Implementation Coverage

Table **??** maps paper claims to actual code.

Table 1: Paper claims vs. implementation status

CRDT Type	Paper	Go Code	Status
G-Counter	✓	✗	**Not implemented**
PN-Counter	✓	✗	**Not implemented**
G-Set	✓	✗	**Not implemented**
OR-Set	✓	✗	**Not implemented**
LWW-Register	✓	✓	Implemented (demo CLI)
RGA Sequence	✗	✗	Not claimed, not implem

The paper's abstract claims "practical implementations for counters, sets, registers, and sequences." Only the LWW-Register exists as runnable Go code. The other four types are described algorithmically in the paper but have zero implementation.

2.2 LWW-Register: Lattice Properties

We constructed encrypted registers with known plaintext values and verified all three lattice laws empirically over FHE ciphertexts.

Table 2: LWW-Register lattice property verification

Property	Test Cases	Result	Notes
Commutativity	(7,3)⊔(12,5)	**PASS**	Both orders yield (12
Idempotence	(9,4)⊔(9,4)	**PASS**	Self-merge preserves
Associativity	3-way merge	**PASS**	(A⊔B)⊔C = A⊔(B⊔

The merge algorithm (MSB-first comparator chain + MUX selection) is correct. After decryption, the encrypted merge result is observation-

ally equivalent to the plaintext LWW merge for all test vectors.

Noise budget. Each MUX gate requires 3 bootstraps (AND, AND, OR). The 4-bit comparator requires $4 \times 2 = 8$ bootstraps (ANDNY + XNOR per bit) plus $3 \times 2 = 6$ bootstraps for the reduction chain (AND + OR + AND per non-MSB bit), totaling 14 comparator bootstraps. The value selection adds $4 \times 3 = 12$ MUX bootstraps and timestamp selection adds another 12, for **38 total bootstraps per 4-bit LWW merge**.

Because each bootstrap resets noise to a fresh level (programmable bootstrapping), there is *no cumulative noise degradation* across chained merges. This is a strength of the TFHE/boolean-gate approach: idempotence is preserved regardless of merge depth k.

2.3 G-Counter: Theoretical Analysis

The paper specifies G-Counter merge as per-replica FHE.max, which decomposes identically to the LWW timestamp comparator. Since the same comparator + MUX circuit is used, the lattice properties hold by the same argument. We verified this using `gcounterMerge` on 5 input pairs including equal values and extremes (0, 15). All passed.

2.4 G-Set: Theoretical Analysis

G-Set merge is encrypted OR of membership flags. OR is commutative, associative, and idempotent over booleans. Since bootstrapping preserves bit correctness, encrypted OR inherits these properties. We verified all 4 truth-table entries plus idempotence explicitly. **PASS.**

2.5 OR-Set: Gap Identified

The paper's OR-Set description (Section 4.4) uses tag-set union for merge, which is correct for the plaintext CRDT. However, there is no encrypted implementation. The Solidity contract (`EncryptedCRDT.sol`) implements OR-Set with *plaintext* tombstone flags and unique tags, only encrypting the element *values*. The add/remove semantics leak set membership patterns.

2.6 Solidity Contract: Privacy Leak

`EncryptedCRDT.sol` line 11: `uint256 timestamp` is stored in plaintext. The `mergeRegisters` function (line 88) performs timestamp comparison in the clear:

```
if (reg1.timestamp > reg2.timestamp)
    {
    merged = reg1.value;
```

This directly contradicts the paper's claim that "merge operations [are performed] homomorphically, keeping data encrypted at all replicas." The timestamp—which reveals write ordering and activity patterns—is fully exposed.

The Go implementation correctly compares timestamps homomorphically. The Solidity contract does not.

3 Empirical Benchmark Results

3.1 Measured Performance

Table 3: fheCRDT merge throughput (PN10QP27, M1 Max, Go 1.26)

Operation	Latency	Ops/sec	Bootst
Single bootstrap (Refresh)	108 ms	9.3	
G-Set merge (1 element)	114 ms	8.8	
G-Counter merge (4-bit, 1 slot)	2,894 ms	0.35	
LWW-Register merge (4-bit)	4,149 ms	0.24	
Geo-convergence (50ms RTT)	4,146 ms	—	

3.2 Comparison to Paper Claims

3.3 FHE vs. Plaintext Overhead

3.4 Ciphertext Size

The paper claims 64 bytes for an LWW-Register state. The measured size is 136 KB—a 2,132× discrepancy. The 64-byte figure may refer to plaintext state size, not encrypted state.

Table 4: Claimed vs. measured performance

Metric	Paper Claim	Measured	Ratio
G-Counter merge (10 replicas)	500 ops/sec	0.035 ops/sec†	**14,286× slower**
LWW-Register merge	330 ops/sec	0.24 ops/sec	**1,375× slower**
G-Set merge (1K elements)	65 ops/sec	0.009 ops/sec‡	**7,222× slower**
Convergence time	450 ms	4,146 ms	**9.2× slower**
LWW state size	64 bytes	136,456 bytes	**2,132× larger**

† Extrapolated: 10 replica slots × 2,894 ms each =∼28.9 sec total.

‡ Extrapolated: 1000 elements × 114 ms each =∼114 sec total.

Table 5: Encrypted vs. plaintext merge latency

CRDT Type	Plaintext (ns)	Encrypted (ns)	Overhead
G-Counter (1 slot)	100	2,894,000,000	∼29 million×
LWW-Register	30	4,149,000,000	∼138 million×
G-Set (1 element)	326	114,000,000	∼350,000×

4 Bugs and Issues

1. **Missing implementations** (`cmd/crdt/main.go`): Only LWW-Register is implemented. The paper claims G-Counter, PN-Counter, G-Set, and OR-Set implementations exist. They do not.

2. **Solidity timestamp leak** (`examples/encrypted-crdt/contracts/EncryptedCRDT.sol:11,57,102`): Timestamps are stored and compared in plaintext, breaking the homomorphic merge claim.

3. **Solidity OR-Set tombstone leak** (`EncryptedCRDT.sol:23`): The `removed` flag is a plaintext `bool`, leaking element removal events.

4. **Paper state size claim** (`main.tex` Table 4, line 372): "LWW-Register: 64 bytes" is incorrect for encrypted state. Actual encrypted size is 136 KB with PN10QP27 parameters.

5. **Paper throughput claims** (`main.tex` Table 4, line 367–373): All ops/sec figures are 3–4 orders of magnitude optimistic. No benchmark code or test environment is documented in the paper. The "500 ops/sec" G-Counter claim would require sub-2ms merge latency; a single bootstrap takes 108ms.

6. **Missing references in paper** (`main.tex:291,316`): References `[zap]` and `[luxcpp-fhe]` are cited but not defined in `references.bib`.

7. **No noise budget monitoring**: The implementation has no mechanism to track remaining noise budget or warn when ciphertexts approach decryption failure. While TFHE bootstrapping resets noise per-gate, corrupted inputs from network transport or storage bit-flips would produce silent wrong answers.

5 Recommendation

The CRDT is not production-ready for Hanzo Base as a sync backend.

The gaps are:

1. **4/5 claimed CRDT types have no implementation.** Only LWW-Register exists. G-Counter, PN-Counter, G-Set, and OR-Set must be built before the framework can replace the plaintext CRDTs in `hanzo/base/crdt/`.
2. **Performance is 3–4 orders of magnitude too slow.** A 10-replica G-Counter merge would take ∼29 seconds. Base requires sub-100ms sync for interactive use. Closing this gap requires either:
   - GPU-accelerated bootstrapping (the paper cites 13–17× speedup, still 2 orders of magnitude short), or

- CKKS/BFV packed ciphertexts for counter operations (avoiding per-bit bootstrapping), or
- Accepting that FHE merges happen asynchronously in batch, with plaintext CRDTs for real-time sync and periodic FHE reconciliation for privacy.

3. **The Solidity contract contradicts the threat model.** Timestamps and tombstones are plaintext. The contract needs encrypted Lamport clocks and encrypted set-membership flags with FHE comparison for merge.
4. **No delta-state support.** The paper describes delta-state fheCRDTs (Section 5.1) but there is no implementation. Full-state sync of 136 KB per LWW-Register per replica is prohibitive at scale.

Path forward. The theoretical framework is sound—lattice properties are preserved under FHE encryption, and the TFHE bootstrapping model avoids noise accumulation. The implementation needs: (a) G-Counter and OR-Set Go implementations using the existing gate primitives; (b) CKKS-mode packed counters for practical throughput; (c) delta-state protocol; (d) the Solidity contract rewritten with encrypted timestamps. Estimated effort: 4–6 engineer-weeks.

Acknowledgments

Benchmarks ran on Apple M1 Max (10-core, 64 GB) under Darwin 25.4.0 with Go 1.26.0. FHE parameters: PN10QP27 (N=1024, Q=134215681), 128-bit classical security level.

References

[1] Z. Kelling, "fheCRDT: Conflict-Free Replicated Data Types over Encrypted State," Lux Industries, 2025.

[2] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "Conflict-free Replicated Data Types," SSS, 2011.

[3] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast Fully Homomorphic Encryption over the Torus," ASIACRYPT, 2016.